

Feature Engineering

Assistant Professor

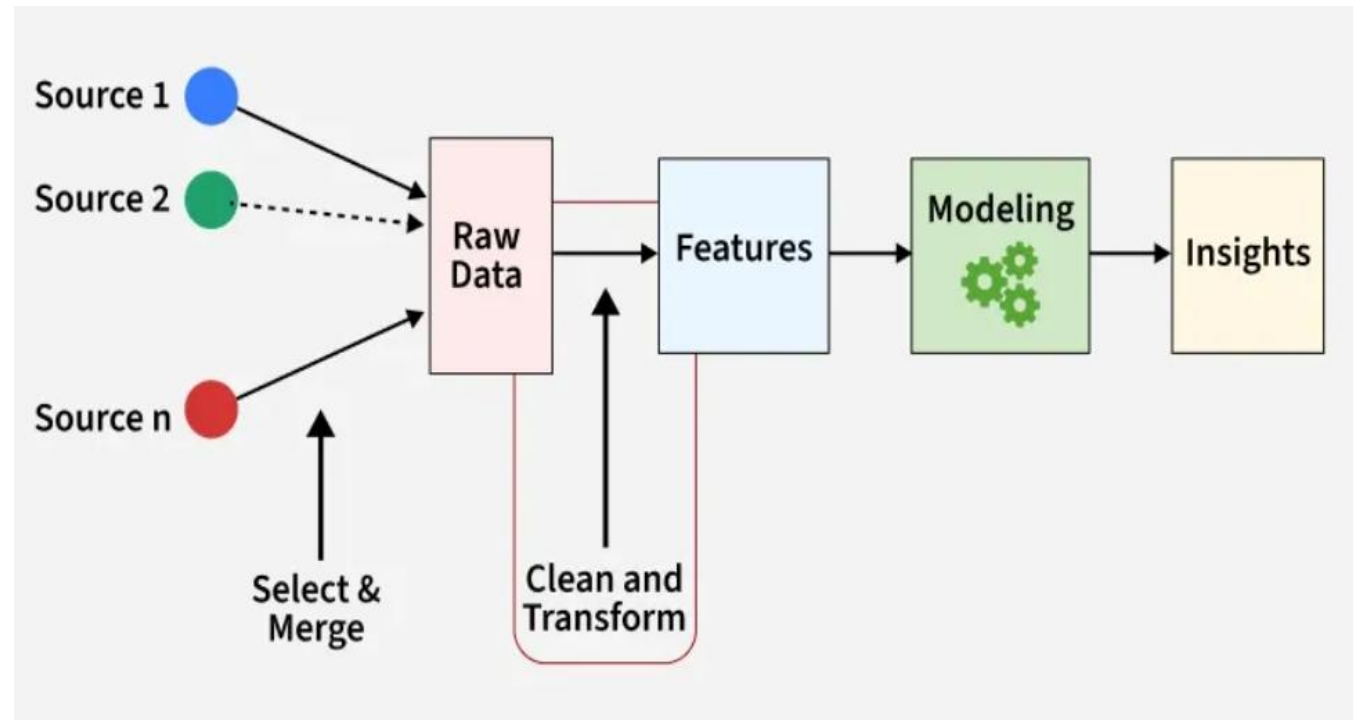
Department of CSE

Introduction

- Feature engineering is the process of transforming raw data into relevant information for use by predictive models.
- In other words, feature engineering is the process of creating predictive model features.
- A feature, also called a dimension, is an input variable used to generate model predictions. Because model performance largely rests on the quality of data used during training.
- Feature engineering is a crucial preprocessing technique that requires selecting the most relevant aspects of raw training data for both the predictive task and model type under consideration.

Introduction

- A feature or attribute is a measurable property of data that is used as input for machine learning algorithms.
- Features can be numerical, categorical, or text-based, representing essential aspects of data relevant to the problem.
- For example, in housing price prediction, features might include the number of bedrooms, location, and property age.



Processes Involved in Feature Engineering

Feature Creation: Feature creation involves generating new features from domain knowledge or by observing patterns in the data. It can be:

Domain-specific: Created based on industry knowledge, like business rules.

Data-driven: Derived by recognizing patterns in data.

Synthetic: Formed by combining existing features.



Feature Creation



Feature Transformation



Feature Extraction



Feature Selection



Feature Scaling

Feature Creation Example

- From a Date field → extract **day, month, year, day of week**.
- From Price and Quantity → create **Total_Sales = Price × Quantity**.
- From Age → create **Age_Group= {Child, Teen, Adult, Senior}**.

Raw Features	New Feature (Created)	Use Case
Quantity, Price	Total_Sales = Quantity × Price	Helps in revenue prediction
Joining_Date, Current_Date	Years_of_Experience = Current_Date – Joining_Date	promotion/salary prediction

Feature Transformation

- Feature transformation is the process of converting one feature type into another, more readable form for a particular model. This involves transforming continuous data into categorical data, or vice versa.
- It is the process of **modifying existing features** into a form that is more suitable for machine learning models. It **improves the distribution** of data, it makes features **comparable** by scaling, and it helps **algorithms converge faster** and perform better.

Techniques for feature transformation

Normalization (Min-Max Scaling): Normalization transforms data to fit within a specific range, typically between 0 and 1. This is achieved by adjusting each feature's values based on its minimum and maximum values. The formula for Min-Max Normalization is:

Normalization (Min-Max Scaling):

- For example, if a feature "Age" ranges from 18 to 80, normalization rescales it so that all values fall between 0 and 1. This ensures that no single feature dominates due to its magnitude.
- Normalization is particularly useful for algorithms that rely on distance measures, such as k-Nearest Neighbours (KNN) or Support Vector Machines (SVM). However, it can be sensitive to outliers, as extreme values can skew the range.
- Mathematical formula for finding the normalized value based on Min-Max Scaling is as follows.

$$X_{Normalized} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

Normalization (Min-Max Scaling):

Example : Suppose a feature named as **Salary** of three employee is given as **salary** = [30,000; 50,000; 80,000]. Find the normalized value of the feature salary using Min Max Scaling.

Given: salary = [30,000; 50,000; 80,000], Min = 30,000, Max = 80,000

We know that

$$\textit{Salary}_{Normalized} = \frac{\textit{Salary} - \textit{Salary}_{min}}{\textit{Salary}_{max} - \textit{Salary}_{min}}$$

For salary = 30,000

$$\frac{30000 - 30000}{80000 - 30000} = \frac{0}{50000} = 0$$

Normalization (Min-Max Scaling):

For salary = 50,000

$$\frac{50000 - 30000}{80000 - 30000} = \frac{20000}{50000} = 0.40$$

For salary = 80,000

$$\frac{80000 - 30000}{80000 - 30000} = \frac{50000}{50000} = 1.00$$

Salary_{Normalized} = [0, 0.4, 1]

Standardization or Scaling

Scaling adjusts the **spread or variability of data**, often centering it around a mean of 0 and scaling it based on standard deviation. This process is also known as standardization. The formula for standardization is:

$$Scaled_{value} = \frac{X - \min(X)}{std(X)}$$

Scaling is particularly useful for algorithms like Principal Component Analysis (PCA) or gradient descent-based models, which assume zero-centered data.

Scaling is less sensitive to outliers compared to normalization and is suitable for features with unbounded ranges. It preserves the relative importance of features while standardizing their scale.

Example

Let us consider a vector $X=[50,60,70]$

Mathematical formula for

$$\text{Standarized Value} = \frac{Xi - \min(X)}{std(X)}$$

$$\mu = \frac{50 + 60 + 70}{3} = \frac{180}{3} = 60$$

$$\begin{aligned}\sigma &= \sqrt{\frac{(50 - 60)^2 + (60 - 60)^2 + (70 - 60)^2}{3}} \\ &= \sqrt{\frac{(-10)^2 + 0^2 + (10)^2}{3}} \\ &= \sqrt{\frac{100 + 0 + 100}{3}} \\ &= \sqrt{\frac{200}{3}} = \sqrt{66.67} \approx 8.165\end{aligned}$$

Standardization or Scaling

- For $X = 50$:

$$Z = \frac{50 - 60}{8.165} = \frac{-10}{8.165} \approx -1.225$$

- For $X = 60$:

$$Z = \frac{60 - 60}{8.165} = 0$$

- For $X = 70$:

$$Z = \frac{70 - 60}{8.165} = \frac{10}{8.165} \approx 1.225$$

$X(\text{Standardized}) = [-1.225, 0, 1.225]$

Log Transformation

- Some models do not handle continuous variables with skewed distributions very well. Applies logarithm to reduce **skewness** in highly skewed data.
- Log normalization is a transformation technique dealing with data that exhibits right-skewed or positively skewed distributions.
- This method involves taking the logarithm of each data point, resulting in a more symmetrical distribution making the data better for modelling. The below described graph will show how log normalization helps in removing the skewed.

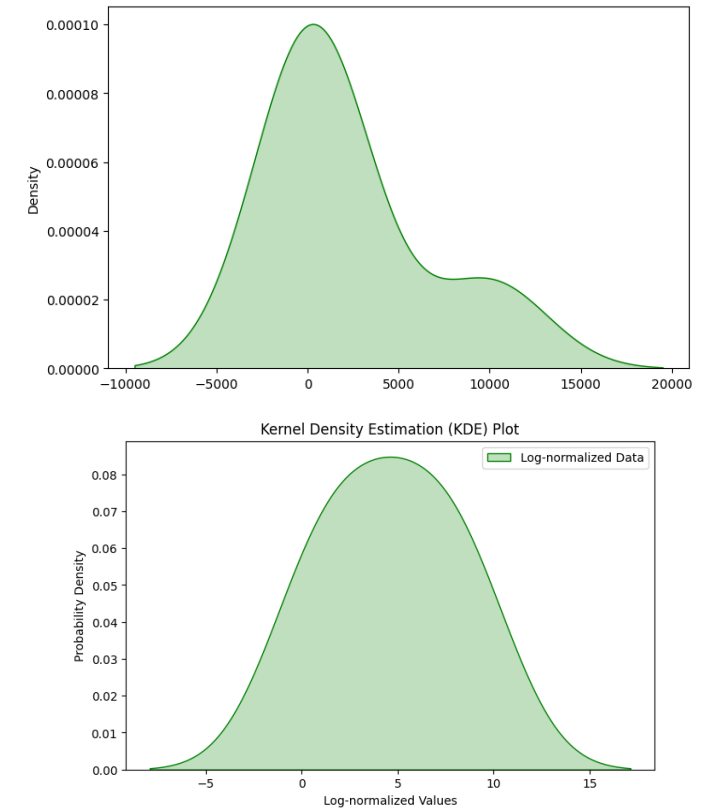
Example:

Sample feature with right-skewed distribution is given as follows

$X = [1, 10, 100, 1000, 10000]$. If we have a KDE plot for the given data it looks like (Given in figure).

Applying log transformation the data distribution looks the (given in the figure).

log normalization is suitable when the data contains only positive values, as the logarithm is not defined for zero or negative values.



Income data = [1000, 2000, 10000]

- $\text{Log}_{10}(1000) = 3$
- $\text{Log}_{10}(2000) \approx 3.3$
- $\text{Log}_{10}(10000) = 4$
- New transformed values = [3, 3.3, 4]

One-Hot Encoding (for categorical variables)

- One-hot encoding is a technique used to convert categorical data into a numerical format that can be used by predictive model.
- It creates a binary column for each category and assigns a value of 1 to the column corresponding to the category and 0 to all other columns.

Original DataFrame:

	Employee	Department
0	A	HR
1	B	Finance
2	C	IT
3	D	Finance

After One-Hot Encoding (pandas):

	Employee	Department_Finance	Department_HR	Department_IT
0	A	0	1	0
1	B	1	0	0
2	C	0	0	1
3	D	1	0	0

Label Encoding

- Label encoding assigns a unique integer to each distinct category for a categorical variable. For example, if a **column** contains the categories "**Red**," "**Green**," and "**Blue**," label encoding might convert them to 0, 1, and 2 respectively.
- It is particularly suitable for ordinal variables where there is a clear, inherent order among the categories, such as education levels (e.g., "High School," "Bachelor's," "Master's") or product ratings (e.g., "Poor," "Average," "Good").

Example:

```
Original categories: ['Red', 'Green', 'Blue', 'Red', 'Green']  
Encoded categories: [2 1 0 2 1]
```


Binning (Discretization)

- Binning, also known as discretization, is a feature transformation technique used in predictive analysis to transform continuous numerical features into discrete, categorical ones.
- This involves dividing the range of a continuous variable into a set of intervals, or "bins," and then assigning each data point to its corresponding bin based on its value.

Purpose of Binning:

Handling Non-linearity:

- Some machine learning algorithms, particularly linear models, may struggle to capture non-linear relationships in the data. Binning can introduce non-linearity by treating different ranges of a continuous variable distinctly.

Binning (Discretization)

- **Reducing Noise and Outlier Impact:**

By grouping values into bins, binning can smooth out minor fluctuations and reduce the influence of extreme outliers, as outliers simply fall into the first or last bin rather than disproportionately affecting the model.

- **Improving Model Interpretability:**

Binned features can sometimes be easier to understand and explain than continuous numerical features, as they represent intuitive categories (e.g., age groups, income brackets).

Example

- Consider an "Age" feature with values ranging from 20 to 70. Instead of using the raw age, you could bin it into categories like:

Young: 20-34, Middle-aged: 35-54, and Senior: 55-70.

- When a model is trained on these binned categories, it learns separate weights or relationships for each age group, potentially capturing non-linear patterns more effectively.

Original Data:

	Age
0	12
1	22
2	35
3	42
4	58
5	65
6	80

After Binning:

	Age	Age_Group
0	12	Teen
1	22	Young
2	35	Young
3	42	Middle Age
4	58	Middle Age
5	65	Senior
6	80	Senior

Code Example

```
import pandas as pd

# Sample dataset
data = {'Age': [12, 22, 35, 42, 58, 65, 80]}
df = pd.DataFrame(data)
print("Original Data:")
print(df)

# Define bin edges
bins = [0, 18, 35, 60, 100] # Ranges: 0-18, 19-35, 36-60, 61-100
# Define bin labels
labels = ['Teen', 'Young', 'Middle Age', 'Senior']

# Apply binning using pd.cut()
df['Age_Group'] = pd.cut(df['Age'], bins=bins, labels=labels)

print("\nAfter Binning:")
print(df)
```

Polynomial transformation

- Polynomial transformation in feature engineering involves creating new features by raising existing features to a specified power or by creating interaction terms between features.
- This allows linear models to capture non-linear relationships within the data.

Generating new features:

For a single feature x , a polynomial transformation of degree k generates new features such as $x^2, x^3, \dots, x^k$. For multiple features, it also includes interaction terms like $x_1x_2, x_1^2x_2$, etc.

Enabling non-linear modeling:

By adding these polynomial terms, a linear model (e.g., Linear Regression, Logistic Regression) can effectively model non-linear patterns in the data, even though the underlying model remains linear in terms of the transformed features.

Feature Extraction

- Feature extraction is the process of identifying and selecting the most important information or characteristics from a data set.
- It's like distilling the essential elements, helping to simplify and highlight the key aspects while filtering out less significant details.
- The feature Extraction technique gives us new features that are a linear combination of the existing features.
- The new set of features will have different values as compared to the original feature values. The main aim is that fewer features will be required to capture the same information

Common Feature Extraction Techniques

- There are often too many factors (features) on the basis of which the final prediction is done.
- The higher the number of features, the harder it gets to visualize the training set and then work on it.
- Sometimes, many of these features are correlated or redundant.

1. Dimensionality reduction

Dimensionality reduction is the process of reducing the number of random features under consideration, by obtaining a set of principal or important features.

Common Feature Extraction Techniques

- By finding a smaller set of new variables, each being a combination of the input variables, containing basically the same information as the input variables.

Two most common method are as follows

- PCA(Principal Component Analysis)
- LDA(Linear Discriminant Analysis)
- Quadratic Discriminant Analysis (QDA)

Common Feature Extraction Techniques

2. Wavelet Transforms

- Useful for extracting features from signals (e.g., audio, images) by analysing different frequency components.

3. Bag of words (BoW)

- The Bag of Words (BoW) model is a foundational and straightforward feature extraction technique predominantly used in Natural Language Processing (NLP).
- In BoW, text (like a sentence or a document) is represented as an unordered collection (or "bag") of its words, disregarding grammar and word order but keeping track of frequency.

Common Feature Extraction Techniques

- Each document is then represented as a numerical vector, where each dimension corresponds to a word in the vocabulary, and the value can be the word count or a binary indicator of presence.
- This approach transforms unstructured text into a structured, numerical format amenable to machine learning algorithms.

4. Image processing techniques

- Feature extraction in image processing involves applying algorithms to raw pixel data to identify and isolate significant visual characteristics, attributes, or patterns within an image.

Common Feature Extraction Techniques

- This can include detecting low-level features like edges, corners, and blobs, or more complex, learned features such as textures, shapes, and object parts using methods like SIFT, SURF, or convolutional layers in Convolutional Neural Networks (CNNs).
- These extracted visual features are then used as input for various computer vision tasks, including image classification, object detection, and image segmentation.

Bag of words(Bow)

Bag of Words (BoW): is a simple yet powerful text representation technique in Natural Language Processing (NLP). Convert text into numerical feature vectors by counting word occurrences while ignoring grammar, order, and context. **How It Works:**

- ① Tokenization: Split the text into words (tokens).
- ② Vocabulary Creation: Create a list of unique words (dictionary) from all documents.
- ③ Vectorization: Convert each document into a vector where each element represents the frequency of a word from the vocabulary.

Bag of words(Bow)

Example: Consider two sentences:

- ① "AI is powerful and efficient."
- ② "AI is efficient but expensive."

The unique words (vocabulary) are:

Vocabulary = {ai, and, but, efficient, expensive, is, powerful}

Each word gets an index:

Word	Index
AI	0
and	1
but	2
efficient	3
expensive	4
is	5
powerful	6

Bag of words(Bow)

Creating the BoW Matrix: Each sentence is represented as a vector where each element indicates the word count at the corresponding index.

Sentence	AI (0)	and (1)	but (2)	efficient (3)	expensive (4)	is (5)	powerful (6)
"AI is powerful and efficient"	1	1	0	1	0	1	1
"AI is efficient but expensive"	1	0	1	1	1	1	0

The Bag of Words (BoW) representation is given by:

$$\begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 & 1 & 1 & 0 \end{bmatrix}$$